

CouchDB

Ratchet wrenches are light and convenient tools you carry around for a range of jobs, big and small. As with power drills, you can swap out variously sized bits like sockets or screws. Unlike a power drill that needs to be plugged into 120 volts of AC power, however, a wrench is happy to rest in your pocket and run on elbow grease. Apache CouchDB is like that. Able to scale down as well as up, it fits problem spaces of varying size and complexity with ease.

CouchDB is the quintessential JSON- and REST-based document-oriented database. First released all the way back in 2005, CouchDB was designed with the web in mind and all the innumerable flaws, faults, failures, and glitches that come with it. Consequently, CouchDB offers a robustness unmatched by most other databases. Whereas other systems tolerate occasional network drops, CouchDB thrives even when connectivity is only rarely available.

Like MongoDB, CouchDB stores *documents*—JSON objects consisting of key-value pairs where values may be any of several types, including other objects nested to any depth. What you don't get, though, is ad hoc querying. Instead, indexed *views* produced by incremental *mapreduce* operations are the principal way you discover documents.

Relaxing on the Couch

CouchDB lives up to its tag line: relax. Instead of focusing on working well only in massive clusters running on hundreds of nodes, CouchDB aims to support a variety of deployment scenarios from the datacenter down to the smartphone. You can run CouchDB on your Android phone, on your MacBook, and in your datacenter. Written in Erlang, CouchDB is built tough—the only way to shut it down is to kill the process! With its append-only storage model, your data is virtually incorruptible and easy to replicate, back up, and restore.

CouchDB is document oriented, using JSON as its storage and communication language. All calls to CouchDB happen through its REST interface. Replication can be one way (from one database to another) or bidirectional (back and forth between databases), and ad hoc (triggered at will) or continuous (triggered at periodic intervals). CouchDB gives you a lot of flexibility to decide how to structure, protect, and distribute your data.

Frontend-to-Backend Syncing with PouchDB

The past few years have seen a dramatic increase in interest in web apps that remain usable even when the user is offline. One of the core difficulties behind building such offline-friendly applications is the data synchronization problem. If a user uses an app offline for an hour and makes a wide variety of changes, what happens when their connection comes back and the discrepancy between client and server state needs to be resolved?

PouchDB is a very interesting tool that has emerged from the CouchDB ecosystem to address this problem. PouchDB is an open source database created in JavaScript that acts as a client-side, in-browser data store that automatically synchronizes data with CouchDB on the backend.

We won't cover PouchDB in this book, but we do find it to be a compelling solution to a problem that's widely encountered but has very few ready-made solutions. If you're looking to use CouchDB as a backend for web apps, you'd be remiss not to take a look.

Comparing CouchDB and MongoDB

One of the big questions we wanted to address in this book is “What’s the difference between CouchDB and MongoDB?” On the surface, CouchDB and MongoDB—which we covered in [Chapter 4, *MongoDB*, on page 93](#)—can seem quite similar. They’re both document-oriented databases with an affinity for JavaScript that use JSON for data transport. There are many differences, though, ranging from project philosophy to implementation to scalability characteristics. We’ll cover many of these as we explore the beautiful simplicity of CouchDB.

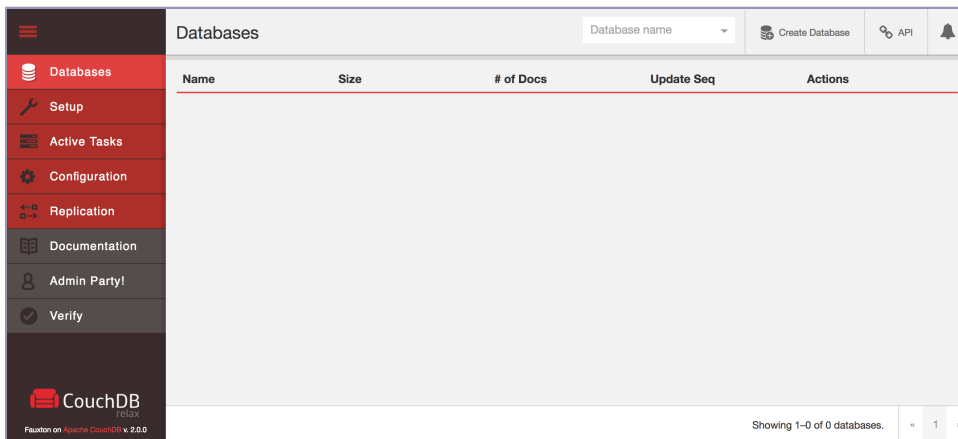
During our three-day tour, we’ll explore many of CouchDB’s compelling features and design choices. We’ll start, as always, with individual CRUD commands and then move on to indexing through mapreduce views. As we’ve done with other databases, we’ll import some structured data and then use it to explore some advanced concepts. Finally, we’ll develop some simple event-driven client-side applications using Node.js and learn how CouchDB’s master-master replication strategy deals with conflicting updates. Let’s get to it!

Day 1: CRUD, Fauxton, and cURL Redux

Today we're going to kick-start our CouchDB exploration by using CouchDB's friendly Fauxton web interface to perform basic CRUD operations. After that, we'll revisit cURL to make REST calls. All libraries and drivers for CouchDB end up sending REST requests under the hood, so it makes sense to start by understanding how they work.

Settling into CouchDB with Fauxton

CouchDB comes with a useful web interface called Fauxton (it was called Futon in pre-2.0 releases). Once you have CouchDB installed and running, open a web browser to `http://localhost:5984/_utils/`. This will open the landing page shown in the figure that follows.

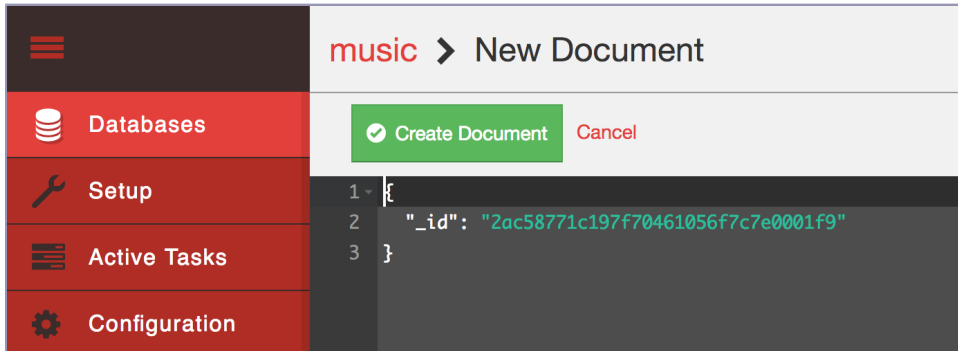


Before we can start working with documents, we need to create a database to house them. We're going to create a database to store data about musicians along with album and track data from those artists' discographies. Click the Create Database button. In the pop-up, enter *music* and click Create. This will redirect you automatically to the database's page. From here, we can create new documents or open existing ones.

Welcome to the Admin Party!

In Fauxton, you may notice the Admin Party! button on the left-hand sidebar navigation panel. You can use this interface to create admin users with specified usernames and passwords. Until you do this, *all* users of your database are admins, which means they can do just about whatever they want, including deleting databases and running expensive replication operations. We won't create an admin user in this chapter, but if you were running CouchDB in production, you would be well advised to do so.

On the music database's page, click the plus sign next to All Documents and then New Doc. This will take you to a new page, as you can see in the figure that follows.



Just as in MongoDB, a document consists of a JSON object containing key-value pairs called *fields*. All documents in CouchDB have an `_id` field, which must be unique and can never be changed. You can specify an `_id` explicitly, but if you don't, CouchDB will generate one for you. In our case, the default is fine, so click Create Document to finish.

Immediately after saving the document, CouchDB will assign it an additional field called `_rev`. The `_rev` field will get a new value every time the document changes. The format for the revision string consists of an integer followed by a dash and then a pseudorandom unique string. The integer at the beginning denotes the numerical revision, in this case 1.

The `_id` and `_rev` field names are reserved in CouchDB. To update or delete an existing document, you must provide *both* an `_id` and the matching `_rev`. If either of these do not match, CouchDB will reject the operation. This is how it prevents conflicts—by ensuring that only the most recent document revisions are modified.

There are no transactions or locking in CouchDB. To modify an existing record, you first read it out, taking note of the `_id` and `_rev`. Then you request an update by providing the full document, including the `_id` and `_rev`. All operations are first come, first served. By requiring a matching `_rev`, CouchDB ensures that the document you think you're modifying hasn't been altered behind your back while you weren't looking.

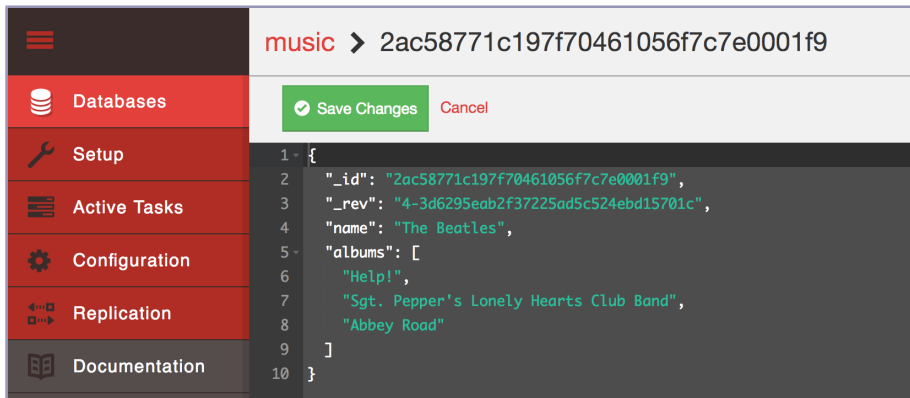
With the document page still open, modify the JSON object, which should have just one `_id`. Enter a key/value pair with a key of *name* and a value of *The Beatles*. Then click the Save Changes button. Your JSON object should look like this:

```
{
  "_id": "2ac58771c197f70461056f7c7e00c0f9",
  "name": "The Beatles"
}
```

CouchDB is not limited to storing string values. It can handle any JSON structure nested to any depth. Modify the JSON again, setting the value of a new *albums* key to the following (this is not an exhaustive list of the Beatles' albums):

```
[
  "Help!",
  "Sgt. Pepper's Lonely Hearts Club Band",
  "Abbey Road"
]
```

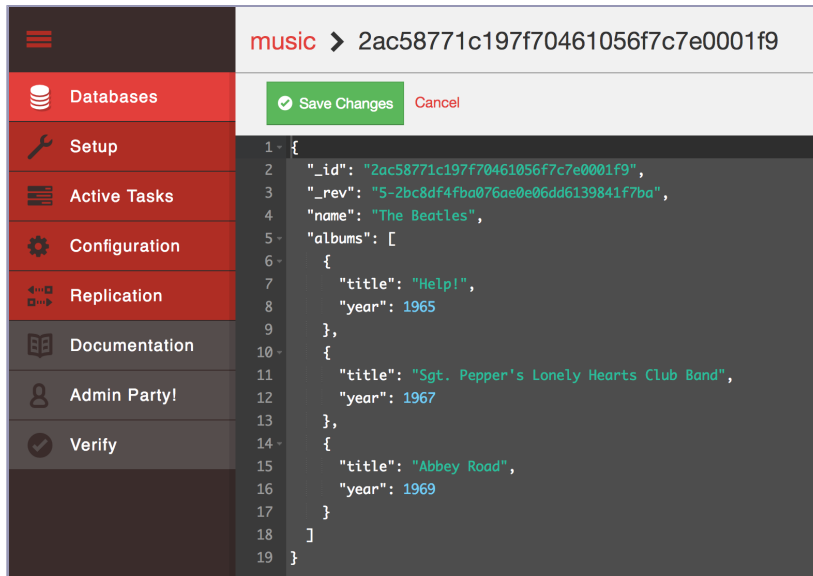
After you click Create Document, it should look like the figure that follows.



There's more relevant information about an album than just its name, so let's add some. Modify the *albums* field and replace the value you just set with this:

```
{
  "title": "Help!",
  "year": 1965
},{
  "title": "Sgt. Pepper's Lonely Hearts Club Band",
  "year": 1967
},{
  "title": "Abbey Road",
  "year": 1969
}]
```

After you save the document, this time you should be able to expand the *albums* value to expose the nested documents underneath. It should resemble the [figure on page 140](#).



Clicking the Delete Document button would do what you might expect; it would remove the document from the music database. But don't do it just yet. Instead, let's drop down to the command line and take a look at how to communicate with CouchDB over REST.

Performing RESTful CRUD Operations with cURL

All communication with CouchDB is REST-based, and this means issuing commands over HTTP. Here we'll perform some basic CRUD operations before moving on to the topic of views. To start, open a command prompt and run the following (which includes setting the root URL for CouchDB as an environment variable for the sake of convenience):

```
$ export COUCH_ROOT_URL=http://localhost:5984
$ curl ${COUCH_ROOT_URL}
{
  "couchdb": "Welcome",
  "version": "2.0.0",
  "vendor": {
    "name": "The Apache Software Foundation"
  }
}
```

Issuing GET requests (cURL's default) retrieves information about the thing indicated in the URL. Accessing the root as you just did merely informs you that CouchDB is up and running and what version is installed. Next, let's get some information about the music database we created earlier (output formatted here for readability):

```
$ curl "${COUCH_ROOT_URL}/music/"
{
  "db_name": "music",
  "update_seq": "4-g1AA...aZxxw",
  "sizes": {
    "file": 24907,
    "external": 193,
    "active": 968
  },
  "purge_seq": 0,
  "other": {
    "data_size": 193
  },
  "doc_del_count": 0,
  "doc_count": 1,
  "disk_size": 24907,
  "disk_format_version": 6,
  "data_size": 968,
  "compact_running": false,
  "instance_start_time": "0"
}
```

This returns some information about how many documents are in the database, how long the server has been up, how many operations have been performed, disk size, and more.

Reading a Document with GET

To retrieve a specific document, append its `_id` to the database URL like so:

```
$ curl "${COUCH_ROOT_URL}/music/2ac58771c197f70461056f7c7e0001f9"
{
  "_id": "2ac58771c197f70461056f7c7e0001f9",
  "_rev": "8-elb7281f6adcd82910c6473be2d4e2ec",
  "name": "The Beatles",
  "albums": [
    {
      "title": "Help!",
      "year": 1965
    },
    {
      "title": "Sgt. Pepper's Lonely Hearts Club Band",
      "year": 1967
    },
    {
      "title": "Abbey Road",
      "year": 1969
    }
  ]
}
```

In CouchDB, issuing GET requests is always safe. CouchDB won't make any changes to documents as the result of a GET. To make changes, you have to use other HTTP commands such as PUT, POST, and DELETE.

Creating a Document with POST

To create a new document, use POST. Make sure to specify a Content-Type header with the value `application/json`; otherwise, CouchDB will refuse the request.

```
$ curl -i -XPOST "${COUCH_ROOT_URL}/music/" \
-H "Content-Type: application/json" \
-d '{ "name": "Wings" }'
```

```
HTTP/1.1 201 Created
Cache-Control: must-revalidate
Content-Length: 95
Content-Type: application/json
Date: Sun, 30 Apr 2017 23:15:42 GMT
Location: http://localhost:5984/music/2ac58771c197f70461056f7c7e002eda
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: 92885ae1d3
X-CouchDB-Body-Time: 0

{
  "ok": true,
  "id": "2ac58771c197f70461056f7c7e002eda",
  "rev": "1-2fe1dd1911153eb9df8460747dfe75a0"
}
```

The HTTP response code 201 Created tells us that our creation request was successful. The body of the response contains a JSON object with useful information, such as the `_id` and `_rev` values.

Updating a Document with PUT

The PUT command is used to update an existing document or create a new one with a specific `_id`. Just like GET, the URL for a PUT URL consists of the database URL followed by the document's `_id`.

```
$ curl -i -XPUT \
"${COUCH_ROOT_URL}/music/2ac58771c197f70461056f7c7e002eda" \
-H "Content-Type: application/json" \
-d '{
  "_id": "74c7a8d2a8548c8b97da748f43000f1b",
  "_rev": "1-2fe1dd1911153eb9df8460747dfe75a0",
  "name": "Wings",
  "albums": ["Wild Life", "Band on the Run", "London Town"]
}'
```

```
HTTP/1.1 201 Created
Cache-Control: must-revalidate
Content-Length: 95
```



```

Content-Type: application/json
Date: Sun, 30 Apr 2017 23:25:13 GMT
ETag: "2-17e4ce41cd33d6a38f04a8452d5a860b"
Location: http://localhost:5984/music/2ac58771c197f70461056f7c7e002eda
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: 6c0bdfffa5
X-CouchDB-Body-Time: 0

{
  "ok": true,
  "id": "2ac58771c197f70461056f7c7e002eda",
  "rev": "2-17e4ce41cd33d6a38f04a8452d5a860b"
}

```

Unlike MongoDB, in which you modify documents *in place*, with CouchDB you always overwrite the entire document to make any change. The Fauxton web interface you saw earlier may have made it look like you could modify a single field in isolation, but behind the scenes it was rerecording the whole document when you hit Save Changes.

As we mentioned earlier, both the `_id` and `_rev` fields must exactly match the document being updated, or the operation will fail. To see how, try executing the same PUT operation again.

```

HTTP/1.1 409 Conflict
Cache-Control: must-revalidate
Content-Length: 58
Content-Type: application/json
Date: Sun, 30 Apr 2017 23:25:52 GMT
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: 5b626b9060
X-CouchDB-Body-Time: 0

{"error":"conflict","reason":"Document update conflict."}

```

You'll get an HTTP 409 Conflict response with a JSON object describing the problem. This is how CouchDB enforces consistency.

Removing a Document with DELETE

Finally, you can use the DELETE operation to remove a document from the database.

```

$ curl -i -XDELETE \
  "${COUCH_ROOT_URL}/music/2ac58771c197f70461056f7c7e002eda" \
  -H "If-Match: 2-17e4ce41cd33d6a38f04a8452d5a860b"
HTTP/1.1 200 OK
Cache-Control: must-revalidate
Content-Length: 95
Content-Type: application/json
Date: Sun, 30 Apr 2017 23:26:40 GMT

```

```
ETag: "3-42aafb7411c092614ce7c9f4ab79dc8b"
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: c4dcb91db2
X-CouchDB-Body-Time: 0

{
  "ok": true,
  "id": "2ac58771c197f70461056f7c7e002eda",
  "rev": "3-42aafb7411c092614ce7c9f4ab79dc8b"
}
```

The DELETE operation will supply a new revision number, even though the document is gone. It's worth noting that the document wasn't really removed from disk, but rather a new empty document was appended, flagging the document as deleted. Just like with an update, CouchDB does not modify documents in place. But for all intents and purposes, it's deleted.

Day 1 Wrap-Up

Now that you've learned how to do basic CRUD operations in Fauxton and cURL, you're ready to move on to more advanced topics. On Day 2, we'll dig into creating indexed *views*, which will provide other avenues for retrieving documents than just specifying them by their `_id` values.

Day 1 Homework

Find

1. Find the CouchDB HTTP API reference documentation online.
2. We've already used GET, POST, PUT, and DELETE. What other HTTP methods are supported?

Do

1. Use cURL to PUT a new document into the music database with a specific `_id` of your choice.
2. Use cURL to create a new database with a name of your choice, and then delete that database also via cURL.
3. CouchDB supports *attachments*, which are arbitrary files that you can save with documents (similar to email attachments). Again using cURL, create a new document that contains a text document as an attachment. Lastly, craft and execute a cURL request that will return just that document's attachment.

Day 2: Creating and Querying Views

In CouchDB, a *view* is a window into the documents contained in a database. Views are the principal way that documents are accessed in all but trivial cases, such as those individual CRUD operations you saw on Day 1. Today, you'll discover how to create the functions that make up a view. You'll also learn how to perform ad hoc queries against views using cURL. Finally, you'll import music data, which will make the views more salient and demonstrate how to use couchrest, a popular Ruby library for working with CouchDB.

Accessing Documents Through Views

A view consists of mapper and reducer functions that are used to generate an ordered list of key-value pairs. Both keys and values can be any valid JSON. The simplest view is called `_all_docs`. It is provided out-of-the-box for all databases and contains an entry for each document in the database, keyed by its string `_id`.

To retrieve all of the things in the database, issue a GET request for the `_all_docs` view.

```
$ curl "${COUCH_ROOT_URL}/music/_all_docs"
{
  "total_rows": 1,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "2ac58771c197f70461056f7c7e0001f9",
      "value": {
        "rev": "7-d37c47883f4d30913c6a38644410685d"
      }
    }
  ]
}
```

You can see in the previous output the one document we've created so far. The response is a JSON object that contains an array of rows. Each row is an object with three fields:

- `id` is the document's `_id`.
- `key` is the JSON key produced by the mapreduce functions.
- `value` is the associated JSON value, also produced through mapreduce.

In the case of `_all_docs`, the `id` and `key` fields match, but for custom views this will almost never be the case.

By default, views will include only metadata for documents in the value field rather than each document's content. To retrieve all of the document's fields, add the `include_docs=true` URL parameter.

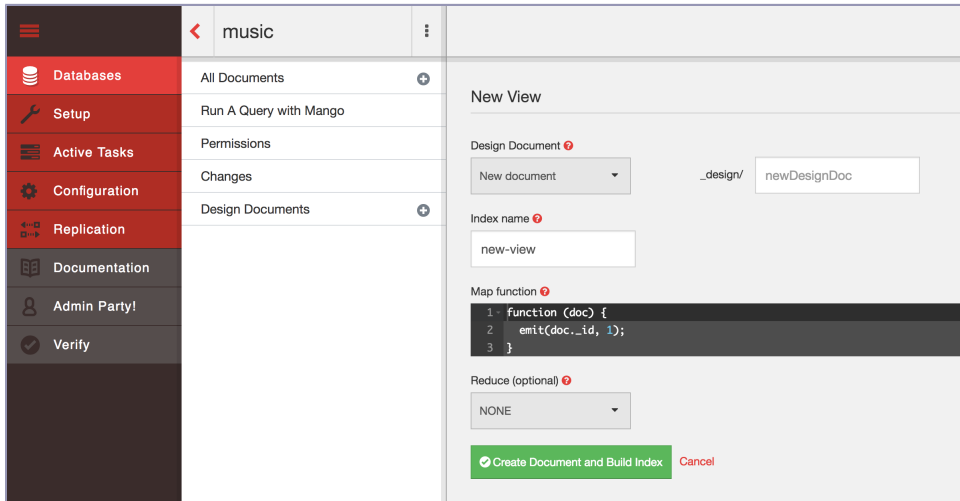
```
$ curl "${COUCH_ROOT_URL}/music/_all_docs?include_docs=true"
{
  "total_rows": 1,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "2ac58771c197f70461056f7c7e0001f9",
      "value": {
        "rev": "7-d37c47883f4d30913c6a38644410685d"
      },
      "doc": {
        "_id": "2ac58771c197f70461056f7c7e0001f9",
        "_rev": "7-d37c47883f4d30913c6a38644410685d",
        "name": "The Beatles",
        "albums": [
          {
            "title": "Help!",
            "year": 1965
          },
          {
            "title": "Sgt. Pepper's Lonely Hearts Club Band",
            "year": 1967
          },
          {
            "title": "Abbey Road",
            "year": 1969
          }
        ]
      }
    }
  ]
}
```

Here you can see that the other properties, name and albums, have been added to the value object in the output. With this basic structure in mind, let's make our own views.

Writing Your First View

Now that we've gotten a rough overview of how views work, let's try creating our own views. To start, we'll reproduce the behavior of the `_all_docs` view, and after that, we'll make increasingly complex views to extract deeper information from our documents for indexing.

To create a view, open a browser to Fauxton¹ as we did in Day 1. Next, open the music database by clicking the link. Click the plus symbol next to Design Documents and then click New View. That should bring you to a page that resembles the figure that follows.



The code in the Map Function box on the right should look like this:

```
function(doc) {
  emit(_doc.id, 1);
}
```

Change the emitter function to `emit(null, doc)`. If you click the Options button in the upper right and then click Run Query, CouchDB will execute this function once for each document in the database, passing in that document as the `doc` parameter each time. This will generate a result that looks like the [figure on page 148](#).

The secret to this output, and all views, is the `emit()` function (this works just like the MongoDB function of the same name). `emit` takes two arguments: the key and the value. A given map function may call `emit` one time, many times, or no times for a given document. In the previous case, the map function emits the key-value pair `null/doc`. As you see in the output table, the key is indeed `null`, and the value is the same object you saw on Day 1 when we requested it directly from `cURL`.

To make a mapper that achieves the same thing as `_all_docs`, we need to emit something a little different. Recall that `_all_docs` emits the document's `_id` field for the key and a simple object containing only the `_rev` field for the value.

1. http://localhost:5984/_utils/

```

id "2ac58771c197f70461056f7c7e0001f9"

{
  "id": "2ac58771c197f70461056f7c7e0001f9",
  "key": null,
  "value": {
    "_id": "2ac58771c197f70461056f7c7e0001f9",
    "_rev": "8-e1b7281f6adcd82910c6473be2d4e2ec",
    "name": "The Beatles",
    "albums": [
      {
        "title": "Help!",
        "year": 1965
      },
      {
        "title": "Sgt. Pepper's Lonely Hearts Club Band",
        "year": 1967
      },
      {
        "title": "Abbey Road",
        "year": 1969
      }
    ]
  },
  "_id": "2ac58771c197f70461056f7c7e0001f9"
}

```

With that in mind, change the Map Function code to the following, and then click Run.

```

function(doc) {
  emit(doc._id, { rev: doc._rev });
}

```

The output table should now resemble the following JSON payload, echoing the same key-value pair we saw earlier when enumerating records via `_all_docs`:

```

{
  "id": "2ac58771c197f70461056f7c7e0001f9",
  "key": "2ac58771c197f70461056f7c7e0001f9",
  "value": {
    "rev": "8-e1b7281f6adcd82910c6473be2d4e2ec"
  },
  "_id": "2ac58771c197f70461056f7c7e0001f9"
}

```

Saving a View as a Design Document

When using CouchDB in production, you should store your views in *design documents*. A design document is a real document in the database, just like the Beatles document we created earlier. As such, it can show up in views and be replicated to other CouchDB servers in the usual fashion. To save a design document in Fauxton, click the Create Document and Build Index button, and then fill in the Design Document and Index name fields.

Design documents always have IDs that start with `_design/` and contain one or more views. The index name distinguishes this view from others housed in the same design document. Deciding which views belong in which design documents is largely application-specific and subject to taste. As a general rule, you should group views based on what they do relative to your data. You'll see examples of this as we create more interesting views.

Finding Artists by Name

Now that we've covered the basics of view creation, let's develop an application-specific view. Recall that our music database stores artist information, including a name field that contains the band's name. Using the normal GET access pattern or the `_all_docs` view, we can access documents by their `_id` values, but we're more interested in looking up bands by name.

In other words, today we can look up the document with `_id` equal to `2ac58771c197f70461056f7c7e0001f9`, but how do we find the document with name equal to *The Beatles*? For this, we need a view. In Fauxton, head back to the New View page, enter *artists* as the name of the design document and *by_name* as the name of the view. Then enter the following Map Function code and click Create Document and Build Index.

```
couchdb/artistsByNameMapper.js
function(doc) {
  if ('name' in doc) {
    emit(doc.name, doc._id);
  }
}
```

This function checks whether the current document has a name field and, if so, emits the name and document `_id` as the relevant key-value pair. This should produce a table like this:

Key	Value
"The Beatles"	"2ac58771c197f70461056f7c7e0001f9"

Finding Albums by Name

Finding artists by name is pretty useful, but we can do more. This time, let's make a view that lets us find albums. This will be the first example where the map function will emit more than one result per document.

Return to the New View page once again (by clicking the plus sign next to Design Documents). Set the design doc name to *albums* and the index name to *by_name*; then enter the following mapper and click Create Document and Build Index:

```
couchdb/albumsByNameMapper.js
function(doc) {
  if ('name' in doc && 'albums' in doc) {
    doc.albums.forEach(function(album){
      var
        key = album.title || album.name,
        value = { by: doc.name, album: album };
      emit(key, value);
    });
  }
}
```

This function checks whether the current document has a name field and an albums field. If so, it emits a key-value pair for each album where the key is the album's title or name and the value is a compound object containing the artist's name and the original album object. The row field should contain an array with these key/value pairs:

Key	Value
"Abbey Road"	{by: "The Beatles", album: {title: "Abbey Road", year: 1969}}
"Help!"	{by: "The Beatles", album: {title: "Help!", year: 1965}}
"Sgt. Pepper's Lonely Hearts Club Band"	{by: "The Beatles", album: {title: "Sgt. Pepper's Lonely Hearts Club Band", year: 1967}}

Just as we did with the Artists By Name view, click the Create Document and Build Index button. This time, for Design Document, enter *albums*, and for the index name enter *by_name*. Click Save to persist the change. Now let's see how to query these documents.

Querying Our Custom Artist and Album Views

Now that we have a couple of custom design documents saved, let's jump back to the command line and query them with the curl command. We'll start with the Artists By Name view. On the command line, execute the following:


```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name"
{
  "total_rows": 1,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "The Beatles",
      "value": "2ac58771c197f70461056f7c7e0001f9"
    }
  ]
}
```

To query a view, construct the path `/<database_name>/_design/<design_doc>/_view/<view_name>`, replacing the parts as appropriate. In our case, we're querying the `by_name` view in the `artists` design document of the `music` database. No surprise here that the output includes our one document, keyed by the band name.

Next, let's try to find albums by name:

```
$ curl "${COUCH_ROOT_URL}/music/_design/albums/_view/by_name"
{
  "total_rows": 3,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Abbey Road",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Abbey Road",
          "year": 1969
        }
      }
    },
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Help!",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Help!",
          "year": 1965
        }
      }
    },
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Sgt. Pepper's Lonely Hearts Club Band",

```

```

    "value": {
      "by": "The Beatles",
      "album": {
        "title": "Sgt. Pepper's Lonely Hearts Club Band",
        "year": 1967
      }
    }
  }
]
}

```

CouchDB will ensure that the records are presented in alphanumeric order by the emitted keys. In effect, this is the indexing that CouchDB offers. When designing your views, it's important to pick emitted keys that will make sense when ordered. Requesting a view in this fashion returns the whole set, but what if you want just a subset? One way to do that is to use the key URL parameter. When you specify a key, only rows with that exact key are returned.

```

$ curl '${COUCH_ROOT_URL}/music/_design/albums/_view/by_name?key="Help!'"
{
  "total_rows": 3,
  "offset": 1,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Help!",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Help!",
          "year": 1965
        }
      }
    }
  ]
}

```

Notice the `total_rows` and `offset` fields in the response. The `total_rows` field counts the total number of records in the view, not just the subset returned for this request. The `offset` field tells us how far into that full set the first record presented appears. Based on these two numbers and the length of the rows, we can calculate how many more records there are in the view on both sides. Requests for views can be sliced a few other ways beyond the keys parameter, but to really see them in action, we're going to need more data.

Importing Data Into CouchDB Using Ruby

Importing data is a recurring problem that you'll face no matter what database you end up using. CouchDB is no exception here. In this section, we'll use

Ruby to import structured data into our music database. Through this, you'll see how to perform bulk imports into CouchDB, and it will also give us a nice pool of data to work with when we create more advanced views.

We'll use music data from Jamendo.com,² a site devoted to hosting freely licensed music. Jamendo provides all their artist, album, and track data in a structured XML format, making it ideal for importing into a document-oriented database such as CouchDB. Download a GZIPped version of the XML:

```
$ curl -O https://imgjam.com/data/dbdump_artistalbumtrack.xml.gz
```

The XML file is over 200 MB. To parse Jamendo's XML file, we'll use the libxml-ruby gem.

Rather than writing our own Ruby-CouchDB driver or issuing HTTP requests directly, we'll use a popular Ruby gem called couchrest that wraps these calls into a convenient Ruby API. We'll be using only a few methods from the API, but if you want to continue using this driver for your own projects, you can check out the documentation.³ On the command line, install the necessary gems:

```
$ gem install libxml-ruby couchrest
```

Just as we did for Wikipedia data in [Chapter 3, HBase, on page 53](#), we'll use a streaming XML parser to process documents sequentially for insert as they're streamed in through standard input. Here's the code:

```
couchdb/import_from_jamendo.rb
require 'libxml'
require 'couchrest'

include LibXML

class JamendoCallbacks
  include XML::SaxParser::Callbacks

  def initialize
    @db = CouchRest.database!("http://localhost:5984/music")
    @count = 0
    @max = 10000 # maximum number to insert
    @stack = []
    @artist = nil
    @album = nil
    @track = nil
    @tag = nil
    @buffer = nil
  end
end
```

2. <http://www.jamendo.com/>

3. <http://rdoc.info/github/couchrest/couchrest/master/>

```

def on_start_element(element, attributes)
  case element
  when 'artist'
    @artist = { :albums => [] }
    @stack.push @artist
  when 'album'
    @album = { :tracks => [] }
    @artist[:albums].push @album
    @stack.push @album
  when 'track'
    @track = { :tags => [] }
    @album[:tracks].push @track
    @stack.push @track
  when 'tag'
    @tag = {}
    @track[:tags].push @tag
    @stack.push @tag
  when 'Artists', 'Albums', 'Tracks', 'Tags'
    # ignore
  else
    @buffer = []
  end
end

def on_characters(chars)
  @buffer << chars unless @buffer.nil?
end

def on_end_element(element)
  case element
  when 'artist'
    @stack.pop
    @artist['_id'] = @artist['id'] # reuse Jamendo's artist id for doc_id
    @artist[:random] = rand
    @db.save_doc(@artist, false, true)
    @count += 1
    if !@max.nil? && @count >= @max
      on_end_document
    end
    if @count % 500 == 0
      puts " #{@count} records inserted"
    end
  when 'album', 'track', 'tag'
    top = @stack.pop
    top[:random] = rand
  when 'Artists', 'Albums', 'Tracks', 'Tags'
    # ignore
  else
    if @stack[-1] && @buffer
      @stack[-1][element] = @buffer.join.force_encoding('utf-8')
      @buffer = nil
    end
  end
end

```

```

    end
  end
end

def on_end_document
  puts "TOTAL: #{@count} records inserted"
  exit(0)
end
end

parser = XML::SaxParser.io(ARGV)
parser.callbacks = JamendoCallbacks.new
parser.parse

```

A few things that you should make note of:

1. At the very beginning of the script, we bring in all of the gems we need.
2. The standard way to use LibXML is by defining a callbacks class. Here we define a `JamendoCallbacks` class to encapsulate our SAX handlers for various events.
3. The first thing our class does during initialization is connect to our local CouchDB server using the `couchrest` API and then create the music database (if it doesn't exist already). After that, it sets up some instance variables for storing state information during the parse. Note that if you set the `@max` parameter to `nil`, all documents will be imported, not just the first 100.
4. Once parsing has started, the `on_start_element()` method will handle any opening tags. Here we watch for certain especially interesting tags, such as `<artist>`, `<album>`, `<track>`, and `<tag>`. We specifically ignore certain container elements—`<Artists>`, `<Albums>`, `<Tracks>`, and `<Tags>`—and treat all others as properties to be set on the nearest container items.
5. Whenever the parser encounters character data, we buffer it to be added as a property to the current container element (the end of `@stack`).
6. Much of the interesting stuff happens in the `on_end_element()` method. Here, we close out the current container element by popping it off the stack. If the tag closes an `<artist>` element, we take the opportunity to save off the document in CouchDB with the `@db.save_doc()` method. For any container element, we also add a random property containing a freshly generated random number. We'll use this later when selecting a random track, album, or artist.
7. Ruby's ARGV stream combines standard input and any files specified on the command line. We feed this into LibXML and specify an instance of our `JamendoCallbacks` class to handle the tokens—start tags, end tags, and character data—as they're encountered.

To run the script, pipe the unzipped XML content into the import script:

```
$ zcat < dbdump_artistalbumtrack.xml.gz | ruby import_from_jamendo.rb
TOTAL: 10000 records inserted
```

The script will begin importing 10,000 records (you can adjust the limit by changing the value of the @max variable in the script). When the import has finished, drop back down to the command line and we'll see how our views look. First, let's pull up a few artists. The limit URL parameter specifies that we want only that number of documents in the response (or less).

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?limit=5"
{"total_rows":10001,"offset":0,"rows":[
{"id":"5385","key":" A.n.K.h // ","value":"5385"},
{"id":"354581","key":" E2U","value":"354581"},
{"id":"457184","key":" EL VECINO","value":"457184"},
{"id":"338059","key":" ENIGMA63","value":"338059"},
{"id":"378976","key":" John 0v3rblast","value":"378976"}
]}
```

The previous request started at the very beginning of the list of artists. To jump to the middle, we can use the startkey parameter:

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?limit=5&startkey=%22C%22"
{"total_rows":10001,"offset":1320,"rows":[
{"id":"363267","key":"C-74","value":"363267"},
{"id":"357962","key":"c-dio","value":"357962"},
{"id":"350911","key":"C-Jay L'infidel J.B","value":"350911"},
{"id":"1188","key":"c-nergy","value":"1188"},
{"id":"832","key":"C. Glen Williams","value":"832"}
]}
```

Previously, we started with artists whose names began with C. Specifying an endkey provides another way to limit the returned content. Here we specify that we want artists only between C and D:

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?startkey=%22C%22&endkey=%22D%22&limit=5"
{"total_rows":10001,"offset":1320,"rows":[
{"id":"363267","key":"C-74","value":"363267"},
{"id":"357962","key":"c-dio","value":"357962"},
{"id":"350911","key":"C-Jay L'infidel J.B","value":"350911"},
{"id":"1188","key":"c-nergy","value":"1188"},
{"id":"832","key":"C. Glen Williams","value":"832"}
]}
```

To get the rows in reverse order, use the descending URL parameter. Be sure to reverse your startkey and endkey as well.

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?|
startkey=%2D%22&endkey=%22C%22&limit=5&descending=true"
{"total_rows":10001,"offset":8174,"rows":[
{"id":"1689","key":"czskamaarù","value":"1689"},
{"id":"341426","key":"CZAQU","value":"341426"},
{"id":"360640","key":"Cystoflo","value":"360640"},
{"id":"355941","key":"CYRUS DA VIRUS","value":"355941"},
{"id":"427004","key":"Cyrrix Project","value":"427004"}
]}
```

A number of other URL parameters are available for modifying view requests, but these are the most common and are the ones you'll reach for the most often. Some of the URL parameters have to do with grouping, which comes from the reducer part of CouchDB mapreduce views. You'll explore these tomorrow.

Day 2 Wrap-Up

Today we covered some good ground. You learned how to create basic views in CouchDB and save them into design documents. You explored different ways of querying views to get subsets of the indexed content. Using Ruby and a popular gem called couchrest, we imported structured data and used it to support our views. Tomorrow, we'll expand on these ideas by creating more advanced views by adding reducers then move on to other APIs that CouchDB supports.

Day 2 Homework

Find

1. We've seen that the `emit()` method can output keys that are strings. What other types of values does it support? What happens when you emit an array of values as a key?
2. Find a list of available URL parameters (like `limit` and `startkey`) that can be appended to view requests and what they do.

Do

1. The import script `import_from_jamendo.rb` assigned a random number to each artist by adding a property called `random`. Create a mapper function that will emit key-value pairs where the key is the random number and the value is the band's name. Save this in a new design document named `_design/random` with the index name `artist`.
2. Craft a cURL request that will retrieve a random artist.

Hint: You'll need to use the `startkey` parameter, and you can produce a random number on the command line using ``ruby -e 'puts rand'``.

3. The import script also added a random property for each album, track, and tag. Create three additional views in the `_design/random` design document with the index names `album`, `track`, and `tag` to match the earlier artist view.

Day 3: Advanced Views, Changes API, and Replicating Data

In Days 1 and 2, you learned how to perform basic CRUD operations and interact with views for finding data. Building on this experience, today we'll take a closer look at views, dissecting the reduce part of the mapreduce equation. After that, we'll develop some Node.js applications in JavaScript to leverage CouchDB's unique Changes API. Lastly, we'll discuss replication and how CouchDB handles conflicting data.

Creating Advanced Views with Reducers

Mapreduce-based views provide the means by which we can harness CouchDB's indexing and aggregation facilities. In Day 2, all our views consisted solely of mappers. Now we're going to add reducers to the mix, developing new capabilities against the data we imported from Jamendo in Day 2.

One great thing about the Jamendo data is its depth. Artists have albums, which have tracks; tracks, in turn, have attributes, including tags. We'll now turn our attention to tags to see whether we can write a view to collect and count them. First, return to the New View page, set the design doc name to *tags* and the index name to *by_name*, and then enter the following map function:

`couchdb/tagsByNameMapper.js`

```
function(doc) {
  (doc.albums || []).forEach(function(album){
    (album.tracks || []).forEach(function(track){
      (track.tags || []).forEach(function(tag){
        emit(tag.idstr, 1);
      });
    });
  });
}
```

This function digs into the artist document and then down into each album, each track, and finally each tag. For each tag, it emits a key-value pair consisting of the tag's `idstr` property (a string representation of the tag, like *"rock"*) and the number 1.

With the map function in place, click on the Reduce selector and choose CUSTOM. Enter the following in the text field:


```
couchdb/simpleCountReducer.js
```

```
function(key, values, rereduce) {
  return sum(values);
}
```

This code merely sums the numbers in the values list—which we’ll talk about momentarily once we’ve run the view. The output should be a series of JSON objects containing key/value pairs like this:

Key	Value
"17sonsrecords"	1
"17sonsrecords"	1
"17sonsrecords"	1
"17sonsrecords"	1
"17sonsrecords"	1
"acid"	1
"acousticguitar"	1
"acousticguitar"	1
"action"	1
"action"	1

This shouldn’t be too surprising. The value is always 1 as we indicated in the mapper, and the Key fields exhibit as much repetition as there is in the tracks themselves. Notice, however, the Options button in the upper-right corner. Check the Reduce box, click Run Query, and then look at the table again. It should now look something like this:

Key	Value
"17sonsrecords"	5
"acid"	1
"acousticguitar"	2
"action"	2
"adventure"	3
"aksband"	1
"alternativ"	1
"alternativ"	3
"ambient"	28
"autodidacta"	17

What happened? In short, the reducer *reduced* the output by combining like mapper rows in accordance with our Reducer Function. The CouchDB mapreduce engine works conceptually like the other mapreducers we’ve seen before (like MongoDB’s [Mapreduce \(and Finalize\), on page 119](#)). Specifically, here’s a high-level outline of the steps CouchDB takes to build a view:

1. Send documents off to the mapper function.
2. Collect all the emitted values.
3. Sort emitted rows by their keys.
4. Send chunks of rows with the same keys to the reduce function.
5. If there was too much data to handle all reductions in a single call, call the reduce function again but with previously reduced values.
6. Repeat recursive calls to the reduce function as necessary until no duplicate keys remain.

Reduce functions in CouchDB take three arguments: key, values, and rereduce. The first argument, key, is an array of tuples—two element arrays containing the key emitted by the mapper and the `_id` of the document that produced it. The second argument, values, is an array of values corresponding to the keys.

The third argument, rereduce, is a Boolean value that will be true if this invocation is a *rereduction*. That is, rather than being sent keys and values that were emitted from mapper calls, this call is sent the products of previous reducer calls. In this case, the key parameter will be null.

Stepping Through Reducer Calls

Let’s work through an example based on the output we just saw. Consider documents (artists) with tracks that have been tagged as “ambient.” The mappers run on the documents and emit key-value pairs of the form “ambient”/1. At some point, enough of these have been emitted that CouchDB invokes a reducer. That call might look like this:

```
reduce(
  [{"ambient", id1}, {"ambient", id2}, ...],    // keys are the same
  [1, 1, ...],                                  // values are all 1
  false                                         // rereduce is false
)
```

Recall that in our reducer function we take the `sum()` of values. Because they’re all 1, the sum will simply be the length—effectively a count of how many tracks have the “ambient” tag. CouchDB keeps this return value for later processing. For the sake of this example, let’s call that number 10.

Some time later, after CouchDB has run these kinds of calls several times, it decides to combine the intermediate reducer results by executing a rereduce:

```
reduce(  
  null,           // key array is null  
  [10, 10, 8],    // values are outputs from previous reducer calls  
  true           // rereduce is true  
)
```

Our reducer function again takes the `sum()` of values. This time, the values add up to 28. Rereduce calls may be recursive. They go on as long as there is reduction to be done, which means until all the intermediate values have been combined into one.

Most mapreduce systems, including the ones used by other databases we've covered in this book such as MongoDB, throw away the output of mappers and reducers after the work is done. In those systems, mapreduce is seen as a means to an end—something to be executed whenever the need arises, each time starting from scratch. Not so with CouchDB.

Once a view is codified into a design document, CouchDB will keep the intermediate mapper and reducer values until a change to a document would invalidate the data. At that time, CouchDB will incrementally run mappers and reducers to correct for the updated data. It won't start from scratch, recalculating everything each time. This is the genius of CouchDB views. CouchDB is able to use mapreduce as its primary indexing mechanism by not tossing away intermediate data values.

Watching CouchDB for Changes

CouchDB's incremental approach to mapreduce is an innovative feature, to be sure; it's one of many that set CouchDB apart from other databases. The next feature we will investigate is the Changes API. This interface provides mechanisms for watching a database for changes and getting updates instantly.

The Changes API makes CouchDB a perfect candidate for a system of record. Imagine a multidatabase system where data is streaming in from several directions and other systems need to be kept up-to-date. Examples might include a search engine backed by Lucene or ElasticSearch or a caching layer implemented using memcached or Redis. You could have different maintenance scripts kick off in response to changes too—performing tasks such as database compaction and remote backups. In short, this simple API opens up a world of possibilities. Today we'll learn how to harness it.

To make use of the API, we're going to develop some simple client applications using Node.js.⁴ Because Node.js is event-driven and code for it is written in JavaScript, it's a natural fit for integrating with CouchDB. If you don't already have Node.js, head over to the Node.js site and install the latest stable version (we use version 7.4.0).

The three flavors of the Changes API are polling, long-polling, and continuous. We'll talk about each of these in turn. As always, we'll start with cURL to get close to the bare metal and then follow up with a programmatic approach.

cURLing for Changes

The first and simplest way to access the Changes API is through the polling interface. Head to the command line, and try the following (the output is truncated for brevity; yours will differ):

```
$ curl "${COUCH_ROOT_URL}/music/_changes"
{
  "results": [{
    "seq": "10057-g1....FqzAI2DMmw",
    "id": "370255",
    "changes": [{"rev": "1-a7b7cc38d4130f0a5f3eae5d2c963d85"}]}
  ], {
    "seq": "10057-g1....A0Y9NEs7RUB",
    "id": "370254",
    "changes": [{"rev": "1-2c7e0deec3ffca959ba0169b0e8bfcef"}]}
  ], {
    ... many more records ...
  }, {
    "seq": "10057-g1....U90zMnILy7J",
    "id": "357995",
    "changes": [{"rev": "1-aa649aa53f2858cb609684320c235aee"}]}
  ],
  "last_seq": 100
}
```

When you send a GET request for `_changes` with no other parameters, CouchDB will respond with everything it has. Just like accessing views, you can specify a `limit` parameter to request just a subset of the data, and adding `include_docs=true` will cause full documents to be returned.

Typically you won't want all the changes from the beginning of time. You're more likely to want the changes that have occurred since you last checked. For this, use the `since` parameter, specifying a sequence ID (pull one from the output of the last cURL command):

4. <http://nodejs.org/>

```
$ curl "${COUCH_ROOT_URL}/music/_changes?since=10057-g1....FqzAI2DMmw"
{
  "results": [{
    "seq": "10057-g1....U90zMnILy7J",
    "id": "357995",
    "changes": [{"rev": "1-aa649aa53f2858cb609684320c235aee"}]
  }],
  "last_seq": 100
}
```

Using this method, the client application would check back periodically to find out whether any new changes have occurred, taking application-specific actions accordingly.

Polling is a fine solution if you can cope with long delays between updates. If updates are relatively rare, then you can use polling without encountering any serious drawbacks. For example, if you were pulling blog entries, polling every five minutes might be just fine.

If you want updates quicker, without incurring the overhead of reopening connections, then long polling is a better option. When you specify the URL parameter `feed=longpoll`, CouchDB will leave the connection open for some time, waiting for changes to happen before finishing the response. Try this:

```
$ curl "${COUCH_ROOT_URL}/music/_changes?feed=longpoll&\n\nsince=10057-g1....FqzAI2DMmw"
{"results": [
```

You should see the beginning of a JSON response but nothing else. If you leave the terminal open long enough, CouchDB will eventually close the connection by finishing it:

```
],
  "last_seq": 9000}
```

From a development perspective, writing a driver that watches CouchDB for changes using polling is equivalent to writing one for long polling. The difference is essentially just how long CouchDB is willing to leave the connection open. Now let's turn our attention to writing a Node.js application that watches and uses the change feed.

Polling for Changes with Node.js

Node.js is a strongly event-driven system, so our CouchDB watcher will adhere to this principle as well. Our driver will watch the changes feed and emit change events whenever CouchDB reports changed documents. To get started, we'll look at a skeletal outline of our driver, talk about the major pieces, and then fill in the feed-specific details.

Without further ado, here's the outline of our watcher program, as well as a brief discussion of what it does:

```
couchdb/watchChangesSkeleton.js
var http = require('http'),
    events = require('events');

/**
 * create a CouchDB watcher based on connection criteria;
 * follows the Node.js EventEmitter pattern, emits 'change' events.
 */
exports.createWatcher = function(options) {

  var watcher = new events.EventEmitter();

  watcher.host = options.host || 'localhost';
  watcher.port = options.port || 5984;
  watcher.last_seq = options.last_seq || 0;
  watcher.db = options.db || '_users';

  watcher.start = function() {
    // ... feed-specific implementation ...
  };

  return watcher;
};

// start watching CouchDB for changes if running as main script
if (!module.parent) {
  exports.createWatcher({
    db: process.argv[2],
    last_seq: process.argv[3]
  })
  .on('change', console.log)
  .on('error', console.error)
  .start();
}
```

So what's happening in this watcher? A few things to pay attention to:

- `exports` is a standard object provided by the CommonJS Module API that Node.js implements. Adding the `createWatcher()` method to `exports` makes it available to other Node.js scripts that might want to use this as a library. The `options` argument allows the caller to specify which database to watch as well as override other connection settings.
- `createWatcher()` produces an `EventEmitter` object that the caller can use to listen for change events. With an `EventEmitter`, you can listen to events by calling its `on()` method and trigger events by calling its `emit()` method.
- `watcher.start()` is responsible for issuing HTTP requests to watch CouchDB for changes. When changes to documents happen, `watcher` should emit

them as change events. All of the feed-specific implementation details will be in here.

- The last chunk of code at the bottom specifies what the script should do if it's called directly from the command line. In this case, the script will invoke the `createWatcher()` method then set up listeners on the returned object that dump results to standard output. Which database to connect to and what sequence ID number to start from can be set via command-line arguments.

So far, there's nothing specific to CouchDB at all in this code. It's all just Node.js's way of doing things. With the skeleton in place, let's add the code to connect to CouchDB via long polling and emit events. The following is just the code that goes inside the `watcher.start()` method. Written inside the previous outline (where the comment says *feed-specific implementation*), the new complete file should be called `watchChangesLongpolling.js`.

```
couchdb/watchChangesLongpollingImpl.js
var httpOptions = {
  host: watcher.host,
  port: watcher.port,
  path: '/' +
    watcher.db +
    '/_changes' +
    '?feed=longpoll&include_docs=true&since=' +
    watcher.last_seq
};

http.get(httpOptions, function(res) {
  var buffer = '';

  res.on('data', function(chunk) {
    buffer += chunk;
  });
  res.on('end', function() {
    var output = JSON.parse(buffer);
    if (output.results) {
      watcher.last_seq = output.last_seq;
      output.results.forEach(function(change) {
        watcher.emit('change', change);
      });
      watcher.start();
    } else {
      watcher.emit('error', output);
    }
  })
})
.on('error', function(err) {
  watcher.emit('error', err);
});
```

Here are some things to look out for in the long polling script:

- The first thing this script does is set up the `httpOptions` configuration object in preparation for the request. The path points to the same `_changes` URL we've been using, with `feed` set to `longpoll` and `include_docs=true`.
- After that, the script calls `http.get()`, a Node.js library method that fires off a GET request according to our settings. The second parameter to `http.get` is a callback that will receive an `HTTPResponse`. The response object emits data events as the content is streamed back, which we add to the buffer.
- Finally, when the response object emits an `end` event, we parse the buffer (which should contain JSON). From this we learn the new `last_seq` value, emit a change event, and then reinvoke `watcher.start()` to wait for the next change.

To run this script in command-line mode, execute it like this (output truncated for brevity):

```
$ node watchChangesLongpolling.js music
{ seq: '...',
  id: '370255',
  changes: [ { rev: '1-a7b7cc38d4130f0a5f3eae5d2c963d85' } ],
  doc:
    { _id: '370255',
      _rev: '1-a7b7cc38d4130f0a5f3eae5d2c963d85',
      albums: [ [Object] ],
      id: '370255',
      name: '""ATTIC""',
      url: 'http://www.jamendo.com/artist/ATTIC_(3)',
      mbgid: '',
      random: 0.4121620435325435 } }
{ seq: '...',
  id: '370254',
  changes: [ { rev: '1-2c7e0deec3ffca959ba0169b0e8bfcef' } ],
  doc:
    { _id: '370254',
      _rev: '1-2c7e0deec3ffca959ba0169b0e8bfcef',
      ... many more entries ...
```

Hurrah, our app works! After outputting a record for each document, the process will keep running, polling CouchDB for future changes.

Feel free to modify a document in Fauxton directly or increase the `@max` value on `import_from_jamendo.rb` and run it again. You'll see those changes reflected on the command line. Next you'll see how to go full steam ahead and use the continuous feed to get even snappier updates.

Watching for Changes Continuously

The polling and long polling feeds produced by the `_changes` service both produce proper JSON results. The *continuous* feed does things a little differently. Instead of combining all available changes into a results array and closing the stream afterward, it sends each change separately and keeps the connection open. This way, it's ready to return more JSON-serialized change notification objects as changes become available.

To see how this works, try the following, supplying a value for `since` (output truncated for readability):

```
$ curl "${COUCH_ROOT_URL}/music/_changes?since=...feed=continuous"
{"seq": "...", "id": "357999", "changes": [{"rev": "1-0329f5c885...87b39beab0"}]}
{"seq": "...", "id": "357998", "changes": [{"rev": "1-79c3fd2fe6...1e45e4e35f"}]}
{"seq": "...", "id": "357995", "changes": [{"rev": "1-aa649aa53f...320c235aee"}]}
```

Eventually, if no changes have happened for a while, CouchDB will close the connection after outputting a line like this:

```
{"last_seq":100}
```

The benefit of this method over polling or long polling is the reduced overhead that accompanies leaving the connection open. There's no time lost reestablishing the HTTP connections. On the other hand, the output isn't straight JSON, which means it's a bit more of a chore to parse. Also, it's not a good fit if your client is a web browser. A browser downloading the feed asynchronously might not receive any of the data until the entire connection finishes (better to use long polling in this case).

Filtering Changes

As you've just seen, the Changes API provides a unique window into the goings-on of a CouchDB database. On the plus side, it provides all the changes in a single stream. However, sometimes you may want just a subset of changes, rather than the fire hose of everything that has ever changed. For example, you may be interested only in document deletions or maybe only in documents that have a particular quality. This is where *filter functions* come in.

A filter is a function that takes in a document (and request information) and makes a decision about whether that document ought to be allowed through the filter. This is gated by the return value. Let's explore how this works. Most artist documents we've been inserting into the music database have a `country` property that contains a three-letter code. Say you're interested only in bands from Russia (RUS). Your filter function might look like the following:

```
function(doc) {
  return doc.country === "RUS";
}
```

If we added this to a design document under the key filters, we'd be able to specify it when issuing requests for `_changes`. But before we do, let's expand the example. Rather than always wanting Russian bands, it'd be better if we could parameterize the input so the country could be specified in the URL.

Here's a parameterized country-based filter function:

```
function(doc, req) {
  return doc.country === req.query.country;
}
```

Notice this time how we're comparing the document's country property to a parameter of the same name passed in the request's query string. To see this in action, let's create a new design document just for geography-based filters and add it:

```
$ curl -XPUT "${COUCH_ROOT_URL}/music/_design/wherabouts" \
-H "Content-Type: application/json" \
-d '{"language":"javascript","filters":{"by_country":
  "function(doc,req){return doc.country === req.query.country;}"
  }}'
{
  "ok":true,
  "id":"_design/wherabouts",
  "rev":"1-c08b557d676ab861957eaeb85b628d74"
}
```

Now we can make a country-filtered changes request:

```
$ curl "${COUCH_ROOT_URL}/music/_changes?|
filter=wherabouts/by_country&|
country=RUS"
{"results":[
  {"seq":10,"id":"5987","changes":[{"rev":"1-2221be...a3b254"}]},
  {"seq":57,"id":"349359","changes":[{"rev":"1-548bde...888a83"}]},
  {"seq":73,"id":"364718","changes":[{"rev":"1-158d2e...5a7219"}]},
  ...
}
```

Because filter functions may contain arbitrary JavaScript, more sophisticated logic can be put into them. Testing for deeply nested fields would be similar to what we did for creating views. You could also use regular expressions for testing properties or compare them mathematically (for example, filtering by a date range). There's even a `userCtx` property on the request object (`req.userCtx`) that you can use to find out more about the credentials provided with the request.

We'll revisit Node.js and the CouchDB Changes API in [Chapter 8, Redis, on page 259](#) when we build a multidatabase application. For now, though, it's time to move on to the last distinguishing feature of CouchDB we're going to cover: replication.

Replicating Data in CouchDB

CouchDB is all about asynchronous environments and data durability. According to CouchDB, the safest place to store your data is on many nodes in your cluster (you can configure how many), and CouchDB provides the tools to do so. Some other databases we've looked at maintain a single master node to guarantee consistency. Still others ensure it with a quorum of agreeing nodes. CouchDB does neither of these; instead, it supports something called multi-master or master-master replication.

Each CouchDB server is equally able to receive updates, respond to requests, and delete data, regardless of whether it's able to connect to any other server. In this model, changes are selectively replicated in one direction, and all data is subject to replication in the same way.

Replication is the last major topic in CouchDB that we'll be discussing. First you'll see how to set up ad hoc and continuous replication between databases. Then you'll work through the implications of conflicting data and how to make applications capable of handling these cases gracefully.

To begin, click the Replication link on the left side of the page. It should open a page that looks like the figure that follows.

In the “Replicate changes from” dialog, choose *music* from the left drop-down menu and enter *music-repl* in the right-side slot. Leave the Continuous checkbox unchecked, check the Create Target box, and then click Replicate. This should produce an event message in the event log below the form followed

by a big green banner at the top of the page saying that the replication operation has begun.

To confirm that the replication request worked, go back to the Fauxton Databases page. There should now be a new database called `music-repl` with the same number of documents as the `music` database. If it has fewer, give it some time and refresh the page—CouchDB may be in the process of catching up. Don't be concerned if the Update Seq values don't match. That's because the original `music` database had deletions and updates to documents, whereas the `music-repl` database had only insertions to bring it up to speed.

Creating Conflicts

Next, we'll create a conflict and then explore how to deal with it. Keep the Replicator page handy because we're going to be triggering ad hoc replication between `music` and `music-repl` frequently. Drop back to the command line, and enter this to create a document in the `music` database:

```
$ curl -XPUT "${COUCH_ROOT_URL}/music/theconflicts" \
-H "Content-Type: application/json" \
-d '{ "name": "The Conflicts" }'
{
  "ok":true,
  "id":"theconflicts",
  "rev":"1-e007498c59e95d23912be35545049174"
}
```

On the Replication page, click Replicate to trigger another synchronization. We can confirm that the document was successfully replicated by retrieving it from the `music-repl` database.

```
$ curl "${COUCH_ROOT_URL}/music-repl/theconflicts"
{
  "_id":"theconflicts",
  "_rev":"1-e007498c59e95d23912be35545049174",
  "name":"The Conflicts"
}
```

Next, let's update it in `music-repl` by adding an album called *Conflicts of Interest*.

```
$ curl -XPUT "${COUCH_ROOT_URL}/music-repl/theconflicts" \
-H "Content-Type: application/json" \
-d '{
  "_id": "theconflicts",
  "_rev": "1-e007498c59e95d23912be35545049174",
  "name": "The Conflicts",
  "albums": ["Conflicts of Interest"]
}'
```

```
{
  "ok":true,
  "id":"theconflicts",
  "rev":"2-0c969fbfa76eb7fcdf6412ef219fcac5"
}
```

And create a conflicting update in music proper by adding a different album: *Conflicting Opinions*.

```
$ curl -XPUT "${COUCH_ROOT_URL}/music/theconflicts" \
-H "Content-Type: application/json" \
-d '{
  "_id": "theconflicts",
  "_rev": "1-e007498c59e95d23912be35545049174",
  "name": "The Conflicts",
  "albums": ["Conflicting Opinions"]
}'
{
  "ok":true,
  "id":"theconflicts",
  "rev":"2-cab47bf4444a20d6a2d2204330fdce2a"
}
```

At this point, both the music and music-repl databases have a document with an `_id` value of `theconflicts`. Both documents are at version 2 and derived from the same base revision (`1-e007498c59e95d23912be35545049174`). Now the question is, what happens when we try to replicate between them?

Fine-Grained Data Replication in CouchDB 2.0

With early versions of CouchDB, if you set up a multi-node CouchDB cluster, *all* documents were stored on all nodes in the cluster. CouchDB 2.0 brought a native clustering capability that enables you to store documents on only *some* nodes in the cluster and to determine, at write time, how many nodes should store the document, enabling you to choose a *replication factor* for data.

We won't cover this or related issues such as sharding, but if you plan on using CouchDB in production, you should check out some theoretical documentation^a to see how it works and receive guidance in making intelligent decisions about replication.

a. <http://docs.couchdb.org/en/latest/cluster/theory.html>

Resolving Conflicts

With our document now in a conflicting state between the two databases, head back to the Replication page and kick off another replication. If you were

expecting this to fail, you may be shocked to learn that the operation succeeds just fine. So how did CouchDB deal with the discrepancy?

It turns out that CouchDB basically just picks one and calls that one the winner. Using a deterministic algorithm, all CouchDB nodes will pick the same winner when a conflict is detected. However, the story doesn't end there. CouchDB stores the unselected “loser” documents as well so that a client application can review the situation and resolve it at a later date.

To find out which version of our document won during the last replication, we can request it using the normal GET request channel. By adding the `conflicts=true` URL parameter, CouchDB will also include information about the conflicting revisions.

```
$ curl "${COUCH_ROOT_URL}/music-repl/theconflicts?conflicts=true"
{
  "_id": "theconflicts",
  "_rev": "2-cab47bf4444a20d6a2d2204330fdce2a",
  "name": "The Conflicts",
  "albums": ["Conflicting Opinions"],
  "_conflicts": [
    "2-0c969fbfa76eb7fcdf6412ef219fcac5"
  ]
}
```

So, we see that the second update won. Notice the `_conflicts` field in the response. It contains a list of other revisions that conflicted with the chosen one. By adding a `rev` parameter to a GET request, we can pull down those conflicting revisions and decide what to do about them.

```
$ curl "${COUCH_ROOT_URL}/music-repl/theconflicts?rev=2-0c969f..."
{
  "_id": "theconflicts",
  "_rev": "2-0c969fbfa76eb7fcdf6412ef219fcac5",
  "name": "The Conflicts",
  "albums": ["Conflicts of Interest"]
}
```

The takeaway here is that CouchDB does not try to intelligently merge conflicting changes. How you should merge two documents is highly application specific, and a general solution isn't practical. In our case, combining the two `albums` arrays by concatenating them makes sense, but one could easily think of scenarios where the appropriate action is not obvious.

For example, imagine you're maintaining a database of calendar events. One copy is on your smartphone; another is on your laptop. You get a text message from a party planner specifying the venue for the party you're hosting, so you

update your phone database accordingly. Later, back at the office, you receive another email from the planner specifying a *different* venue. So you update your laptop database and then replicate between them.

CouchDB has no way of knowing which of the two venues is "correct." The best it can do is make them consistent, keeping the old value around so you can verify which of the conflicting values should be kept. It would be up to the application to determine the right user interface for presenting this situation and asking for a decision.

Day 3 Wrap-Up

And so ends our tour of CouchDB. Here in Day 3, you started out by learning how to add reducer functions to your mapreduce-generated views. After that, we took a deep dive into the Changes API, including a jaunt into the world of event-driven server-side JavaScript development with Node.js. Lastly, we took a brief look at how you can trigger replication between databases and how client applications can detect and correct for conflicts.

Day 3 Homework

Find

1. What native reducers are available in CouchDB? What are the benefits of using native reducers over custom JavaScript reducers?
2. How can you filter the changes coming out of the `_changes` API on the server side?
3. Like everything in CouchDB, the tasks of initializing and canceling replication are controlled by HTTP commands under the hood. What are the REST commands to set up and remove replication relationships between servers?
4. How can you use the `_replicator` database to persist replication relationships?

Do

1. Create a new module called `watchChangesContinuous.js` based on the skeletal Node.js module described in the section [Polling for Changes with Node.js, on page 163](#).
2. Implement `watcher.start()` such that it monitors the continuous `_changes` feed. Confirm that it produces the same output as `watchChangesLongpolling.js`.

Hint: If you get stuck, you can find an example implementation in the downloads that accompany this book.

3. Documents with conflicting revisions have a `_conflicts` property. Create a view that emits conflicting revisions and maps them to the `doc_id`.

Wrap-Up

Throughout this chapter, you've seen how to do a pretty wide range of tasks with CouchDB, from performing basic CRUD operations to building views out of mapreduce functions. You saw how to watch for changes, and you explored how to develop nonblocking event-driven client applications. Finally, you learned how to perform ad-hoc replication between databases and how to detect and resolve conflicts. Despite all of this content, there's still a lot we didn't cover, but now it's time to wrap things up before heading off to our next database.

CouchDB's Strengths

CouchDB is a robust and stable member of the NoSQL community. Built on the philosophy that networks are unreliable and hardware failure is imminent, CouchDB offers a heartily decentralized approach to data storage. Small enough to live in your smartphone and big enough to support the enterprise, CouchDB affords a variety of deployment situations.

CouchDB's Weaknesses

Of course, CouchDB isn't well suited for everything. CouchDB's mapreduce-based views, while novel, can't perform all the fancy data slicing you'd expect from a relational database. In fact, you shouldn't be running ad-hoc queries *at all* in production. As with many other NoSQL databases, CouchDB always works best when you have a very good sense of what you're going to need in advance. In some databases, that means knowing the key or "address" of an object; in CouchDB, that means knowing all of your queries in advance. If your use case doesn't allow for that kind of foreknowledge, then you may want to look elsewhere.

Parting Thoughts

CouchDB's attention to robustness in the face of uncertainty makes it a great choice if your system must stand up to the harsh realities of the wild Internet. By leveraging standard "webisms" like HTTP/REST and JSON, CouchDB fits in easily wherever web technologies are prevalent, which is increasingly everywhere. Inside the walled garden of a datacenter, CouchDB can still make sense if you commit to managing conflicts when they arise or if you utilize some more recently added clustering features.

There are plenty of other features that make CouchDB unique and special that we didn't have time to cover. A short list would include ease of backups, binary attachments to documents, and CouchApps, a system for developing and deploying web apps directly through CouchDB with no other middleware. Having said that, we hope we've provided enough of an overview to whet your appetite for more. Try CouchDB for your next data-driven web app; you won't be disappointed!